

```

# -*- coding: utf-8 -*-
"""Bateria v3 – Método Trinário
(apresentação) vs Lógica Trinária E? v2.0.
Modelo A (v2.0): balanço único  $B = \sum w e / \sum w$ ,
massa  $M = \sum w$ , limiar  $\theta$ , massa mínima  $m_0$ .
Modelo B (apresentação): médias separadas
 $M^+/M^-$ , cobertura  $C_{\pm}$ , suporte  $E_{\pm} = M_{\pm} \cdot C_{\pm}$ ,
resultante  $R = E^+ - E^-$ , conflito  $K =$ 
 $2 \cdot \min(E^+, E^-) / (E^+ + E^-)$ , valor final  $v = R \cdot s$ ,
escala com teto  $\pm 0,95$ , diagnósticos  $D_0 - D_5$ .
Instanciação necessária para testar (o doc
deixa cobertura qualitativa):
 $C(m) = m / (m+1)$  – saturante:  $C(0)=0$ , cresce
com a massa, tende a 1.
"""

import random, json
from itertools import product

CAP = 0.95
def clamp(x): return max(-CAP, min(CAP, x))

# ----- Modelo A (v2.0) -----
-----
def modelo_A(evs, theta=1/3, m0=1.0):
    M = sum(w for w, _ in evs)
    B = (sum(w*e for w, e in evs)/M) if M>0
    else 0.0
    if B>=theta and M>=m0: return dict(B=B,
M=M, valor='V')
    if B<=-theta and M>=m0: return

```

```

dict(B=B, M=M, valor='F')
    return dict(B=B, M=M, valor='E', diag=
('D' if M<m0 else 'C'))

# ----- Modelo B (apresentação)
-----
def cobertura(m): return m/(m+1.0)

def modelo_B(evs, s=1.0, theta=1/3):
    pos = [(w,e) for w,e in evs if e>0]
    neg = [(w,-e) for w,e in evs if e<0]
# módulos das contrárias
    mp = sum(w for w,_ in pos); mn = sum(w
for w,_ in neg)
    Mp = (sum(w*e for w,e in pos)/mp) if
mp>0 else 0.0
    Mn = (sum(w*e for w,e in neg)/mn) if
mn>0 else 0.0
    Cp, Cn = cobertura(mp), cobertura(mn)
    Ep, En = Mp*Cp, Mn*Cn
    K = (2*min(Ep,En)/(Ep+En)) if (Ep+En)>0
else 0.0
    R = Ep - En
    v = clamp(R*s)
    if v>=theta: val='V'
    elif v<=-theta: val='F'
    else: val='E'
    # diagnóstico computável D1-D3
    diag=None
    if val=='E':
        if len(evs)==0: diag='D1
(ausência)'
        elif K>=0.5: diag='D3 (conflito)'
        else: diag='D2 (insuficiência)'

```

```

        return
dict(Mp=round(Mp,3),Mn=round(Mn,3),Cp=round
(Cp,3),Cn=round(Cn,3),

Ep=round(Ep,3),En=round(En,3),R=round(R,3),
K=round(K,3),

v=round(v,3),valor=val,diag=diag)

R_ = {}

#
=====
=====
# TESTE 1 – Verificação do exemplo numérico
da apresentação
# (engenharia reversa do "valor final")
#
=====
=====
Mp,Cp,Mn,Cn,s = 0.813,0.85,0.509,0.45,0.65
Ep,En = Mp*Cp, Mn*Cn
Rr = Ep-En
K = 2*min(Ep,En)/(Ep+En)
R_['exemplo_apresentacao'] = dict(
    E_pos=round(Ep,3), E_pos_doc=0.691,
    E_neg=round(En,3), E_neg_doc=0.229,
    R=round(Rr,3), R_doc=0.462,
    K=round(K,3), K_doc=0.50,
    valor_final_R_vezes_s=round(Rr*s,3),
    valor_final_doc=0.30,
    formula_recuperada='v = R × s
(suficiência conservadora)')

```

```

#
=====
=====
# TESTE 2 – Cenários comparativos A vs B
#
=====
=====
cenarios = {
    'S1 Quantidade × qualidade: 100 fracas a
favor (0.2) vs 1 forte contra (0.9)':
        [(1.0,0.2)]*100 + [(1.0,-0.9)],
    'S2 Evidência única forte (w=0.1, e=+1)':
        [(0.1,1.0)],
    'S3 Conflito simétrico forte': [(2.0,0.9),
(2.0,-0.9)],
    'S4 Sem evidência': [],
    'S5 Consistente forte a favor':
        [(1.0,0.9),(2.0,0.85),(1.0,0.8)],
    'S6 Muitas fracas a favor, nada contra (50
× 0.15)': [(1.0,0.15)]*50,
}
R_['cenarios'] = {}
for nm, evs in cenarios.items():
    A = modelo_A(evs); B = modelo_B(evs)
    R_['cenarios'][nm] = dict(
        A=dict(B=round(A['B'],3),
M=round(A['M'],2), saida=A['valor']+
('/'+A.get('diag','') if A.get('diag') else
'')),
        B=dict(R=B['R'], K=B['K'],
v=B['v'], saida=B['valor']+('/'+B['diag']
if B['diag'] else '')))
#

```

```

=====
=====
# TESTE 3 – Propriedades do índice de
conflito K
#
=====
=====
rand = random.Random(5)
ok_range=True; ok_sim=True;
ok_extremos=True
for _ in range(20000):
    a,b = rand.uniform(0,1),
rand.uniform(0,1)
    if a+b==0: continue
    K1 = 2*min(a,b)/(a+b)
    if not (0<=K1<=1): ok_range=False
    if abs(K1 - 2*min(b,a)/(b+a))>1e-12:
ok_sim=False
K_igual = 2*min(0.7,0.7)/(1.4)          #
E+=E- → K=1
K_unilateral = 2*min(0.7,0.0)/(0.7)    #
unilateral → K=0
ok_extremos = (K_igual==1.0 and
K_unilateral==0.0)
R['propriedades_K'] =
dict(intervalo_0_1=ok_range,
simetrico=ok_sim,
    K_max_em_empate=K_igual,
K_min_unilateral=K_unilateral,
extremos_ok=ok_extremos)

#
=====
=====

```

```

# TESTE 4 – Teto ±0,95: fechamento e
revisabilidade
#
=====
=====
# 4a fechamento algébrico
rand = random.Random(9); fech_ok=True
for _ in range(50000):
    a,b = rand.uniform(-CAP,CAP),
rand.uniform(-CAP,CAP)
    for r in (-a, min(a,b), max(a,b)):
        if not (-CAP-1e-12 <= r <= CAP+1e-
12): fech_ok=False
R['teto_fechamento'] = fech_ok
# 4b sem estado absorvente: do teto, 1
evidência contrária sempre move o valor
evs = [(10.0, 0.95)] # massa alta no
teto
B0 = modelo_A(evs)['B']
B1 = modelo_A(evs+[(1.0,-0.9)])['B']
R['teto_revisabilidade'] =
dict(balanco_no_teto=round(B0,3),
    apos_1_contraria=round(B1,3),
moveu=B1<B0)

#
=====
=====
# TESTE 5 – Conectivos graduados no
contínuo [-0,95, +0,95]
#
=====
=====
rand = random.Random(13)

```

```

ass=com=dm=dist=True
for _ in range(50000):
    a,b,c = (rand.uniform(-CAP,CAP) for _
in range(3))
    if abs(min(min(a,b),c)-
min(a,min(b,c)))>1e-12: ass=False
    if min(a,b)!=min(b,a): com=False
    if abs(-(min(a,b)) - max(-a,-b))>1e-12:
dm=False
    if abs(min(a,max(b,c)) -
max(min(a,b),min(a,c)))>1e-12: dist=False
R_['conectivos_contínuos'] =
dict(associatividade=ass,
comutatividade=com,

de_morgan=dm, distributividade=dist)

#
=====
=====
# TESTE 6 – Modus Ponens graduado: regra
ingênua vs protegida
#
=====
=====
def mp_ingenuo(p, regra): return min(p,
regra)
def mp_protegido(p, regra): return min(p,
regra) if (p>0 and regra>0) else 0.0

casos_mp = {
    'Premissa forte, regra forte (0.8, 0.9)':
(0.8,0.9),
    'Premissa fraca, regra forte (0.3, 0.9)':

```

```

(0.3,0.9),
    'Premissa ZERO, regra forte (0, 0.9)':
(0.0,0.9),
    'Premissa REFUTADA, regra forte (-0.7,
0.9)': (-0.7,0.9),
    'Premissa forte, regra refutada (0.8,
-0.6)': (0.8,-0.6),
}
R_['mp_graduado'] = {}
for nm,(pv,rv) in casos_mp.items():
    R_['mp_graduado'][nm] =
dict(ingenuo=round(mp_ingenuo(pv,rv),3),

protegido=round(mp_protegido(pv,rv),3))
# propriedades do MP protegido
rand=random.Random(17); nao_amp=True;
bloq_zero=True; nega_antec=True
for _ in range(50000):
    pv,rv = rand.uniform(-CAP,CAP),
rand.uniform(-CAP,CAP)
    con = mp_protegido(pv,rv)
    if con>0 and (con>pv+1e-12 or
con>rv+1e-12): nao_amp=False
    if pv<=0 and con!=0: bloq_zero=False
R_['mp_protegido_props'] =
dict(nao_amplificacao=nao_amp,
    bloqueio_por_zero_e_negativo=bloq_zero,
    obs='ingênuo comete negação do
antecedente: premissa -0,7 → conclui
-0,7')

#
=====
=====

```



```

# TESTE 7 – Equivalência por proximidade vs
bicondicional de Kleene
#
=====
=====
def iff_kleene(a,b): return min(max(-a,b),
max(-b,a))
def eq_prox(a,b): return clamp(CAP - abs(a-
b))
casos_eq = {
'Ambas verdadeiras (0.95, 0.95)':
(0.95,0.95),
'Opostas (0.95, -0.95)': (0.95,-0.95),
'Ambas DESCONHECIDAS (0, 0)': (0.0,0.0),
'Uma decidida, outra zero (0.9, 0)':
(0.9,0.0),
}
R_['equivalencia'] = {}
for nm,(a,b) in casos_eq.items():
    R_['equivalencia'][nm] =
dict(proximidade=round(eq_prox(a,b),3),

kleene=round(iff_kleene(a,b),3))
# violação de bloqueio por zero:
eq_prox(0,0)=0.95 afirmaria equivalência
entre 2 incógnitas
R_['equivalencia_veredito'] = dict(
    proximidade_viola_bloqueio_zero=
(eq_prox(0,0)>0),
    kleene_respeita=(iff_kleene(0,0)==0.0),
    recomendacao='↔ lógico = Kleene;
proximidade renomeada como medida de
similaridade  $\text{sim}(\varphi,\psi)$ , fora do cálculo')

```

```

#
=====
=====
# TESTE 8 – Peso multiplicativo:
propriedade de veto e encolhimento
#
=====
=====
fat = [0.8]*6
prod = 1.0
for f in fat: prod*=f
gm = prod**(1/6)
R_['peso_multiplicativo'] = dict(
    veto=('fator 0 anula o peso: OK' ),
    seis_fatores_08=round(prod,3),
    media_geometrica_08=round(gm,3),
    obs='produto de 6 fatores 0,8 → 0,262:
    encolhimento sistemático; média geométrica
    preserva escala (0,8) mantendo o veto')

#
=====
=====
# TESTE 9 – Núcleo trivalente herdado
(projeção por regiões)
# conservatividade e regras já verificadas
na v2.0 por isomorfismo:
# projeção sinal-com-limiar de [-0.95,0.95]
→ {F,E,V} = mesmo núcleo K3
#
=====
=====
def proj(v,theta=1/3): return 'V' if
v>=theta else ('F' if v<=-theta else 'E')

```

```

# comutação: proj(min(a,b)) == AND3(proj a,
proj b)? min/max comutam com projeção
monotônica APENAS
# se limiares consistentes; teste empírico:
AND3 = {('V','V'):'V',('V','E'):'E',
('V','F'):'F',('E','V'):'E',('E','E'):'E',
        ('E','F'):'F',('F','V'):'F',
('F','E'):'F',('F','F'):'F'}
rand=random.Random(23); comuta_falhas=0;
n=50000
for _ in range(n):
    a,b = rand.uniform(-CAP,CAP),
rand.uniform(-CAP,CAP)
    if proj(min(a,b)) !=
AND3[(proj(a),proj(b))]: comuta_falhas+=1
R_['projecao_comuta_com_min'] =
dict(testes=n, falhas=comuta_falhas,
      obs='min/max são monotônicos: compor no
contínuo e projetar = projetar e compor no
trivalente')

print(json.dumps(R_, ensure_ascii=False,
indent=1))

```